



第11章 数据共享与保护

张仁斌@合肥工业大学软件学院

本章主要内容



11.1 数据共享与保护：鱼和熊掌兼得



11.2 变量/对象的作用域与可见性（空间）

11.3 变量/对象的生存期（时间）

11.4 静态变量/类的静态成员

11.5 类的友元

11.6 共享数据的保护

11.7 多文件结构和编译预处理命令

11.8 作业与实践



- ◆ C++采取多种措施增加数据的**安全性**，如类的私有成员，但是某些数据需要被共享
- ◆ 数据能在**一定范围内共享**，又要使它**不被任意修改**——数据的共享与保护
 - 共享数据有时会因误操作等非法改变，既要使数据能在一定范围内共享，又要使它不被任意修改

本章主要内容



11.1 数据共享与保护：鱼和熊掌兼得

11.2 变量/对象的作用域与可见性（空间）



11.3 变量/对象的生存期（时间）

11.4 静态变量/类的静态成员

11.5 类的友元

11.6 共享数据的保护

11.7 多文件结构和编译预处理命令

11.8 作业与实践

变量/对象定义位置



◆ 定义变量或对象的位置

- 函数体内
- 类体内
- 函数原型参数表内
- 所有函数体和类之外

◆ 定义变量或对象的位置不同，其作用域、可见性和生存期不同

◆ 根据作用域，变量或对象分为

- 局部变量/对象
- 全局变量/对象



五种典型作用域

◆ (1) 函数原型形参的作用域 (局部变量/对象)

```
#include <iostream>
using namespace std;
```

```
int max(int x, int y); // 等价于: int max(int, int);
```

x和y的作用域仅限于此，不能用于
程序代码其他地方，因此可有可无

```
int main()
{
    int a=10, b=20;
    cout << max(a, b) << endl;
    return 0;
}
```

```
int max(int x, int y)
{
    return x > y ? x : y;
}
```



◆ (2) 函数形参、函数体内定义的变量/对象，只在函数内有效，函数外部不能使用（**局部变量/对象**）

■ 不同函数中可以使用同名变量/对象，由于作用域不同，不会发生冲突

```
void func(int a) {  
    int b = a;  
    cin >> b;  
    if (b > 0)  
    {  
        int c;  
        .....  
    }  
    .....  
}
```

Diagram illustrating scope boundaries with arrows pointing from labels to the corresponding code blocks:

- a的作用域 (a's scope) points to the `func` function body.
- b的作用域 (b's scope) points to the block `int b = a; cin >> b; if (b > 0) {`.
- c的作用域 (c's scope) points to the innermost block `{ int c; ..... }`.

```
int f1(int x) {  
    int y, a;  
    .....  
}  
  
int f2(int x, int y) {  
    int b;  
    .....  
}  
  
main() {  
    int a, b, x, y, m;  
    .....  
}
```

Diagram illustrating scope boundaries with arrows pointing from labels to the corresponding code blocks:

- x、y、a的作用域仅限于函数f1()内 (Scope of x, y, a is limited to function f1()).
- x、y、b的作用域仅限于函数f2()内 (Scope of x, y, b is limited to function f2()).
- a、b、x、y、m的作用域仅限于main函数内，与前面的a、b、x、y互相独立 (Scope of a, b, x, y, m is limited to the main function, and they are independent of the a, b, x, y defined in the previous functions).



◆不同函数中同名变量/对象

```
#include <iostream>
using namespace std;

void sub()
{
    int a = 6, b = 7;
    cout << "in sub( ):\n";
    cout << a << " " << b << '\n';
    cout << &a << " " << &b << '\n';
}

int main() {
    int a = 3, b = 4;
    cout << a << " " << b << '\n';
    cout << &a << " " << &b << '\n';
    sub();
    return 0;
}
```

◆for循环作用域

```
void function1( ) {
    int x = 1;
    int y = 1;
    for(int i = 1; i < 10; i++){
        x += i;
    }
    for(int i = 1; i < 10; i++){
        y += i;
    }
    cout << x << "\n" << y << "\n";
    cout << i; ✗
}
```

D:\Edu-CPP\chap11\ex11-01.exe

```
3 4
0x70fe0c 0x70fe08
in sub( ):
6 7
0x70fdb8 0x70fdb8
```




◆ (3) 类作用域 (局部)

- 类内：类的成员具有类作用域，在类的范围内可使用，成员函数可以直接访问
- 类外：在类作用域外访问类成员，则通过类对象，只能访问其公有成员

```
class Demo {  
    int a;  
public:  
    Demo(int x, int y) { a = x, b = y; }  
    int b;  
    void show( ) { cout << a << endl; }  
    void add( ) { cout << a + b << endl; }  
};  
  
int main() {  
    Demo x(5, 6);  
    x.show();  
    x.add();  
    cout << x.b;  
    return 0;  
}
```

尽管b的声明在本构造函数的后面，但仍可访问



◆ (4) 文件作用域 (全局变量)

■ 不在前述各个块中声明的标识符，被称为文件作用域，其作用域始于声明点，终于文件尾，称之为全局变量

```
#include<iostream>
using namespace std;
```

```
int i;
int main() {
    i = 5;
    {
        int i = 7;
        cout << "局部i = " << i << ", 全局i = " << ::i << endl;
    }
    cout << "i = " << i << endl;
    return 0;
}
```

两个嵌套的作用域，如果在内层声明了与外层同名的标识符，则外层作用域的标识符在内层不可见

全局变量与局部变量同名，在局部局部变量起作用，如果想在局部操作全局同名变量，可利用作用域运算符::

```
> D:\Edu-CPP\chap11\ex11-02.exe
```

```
局部 i = 7, 全局 i = 5
i = 5
```

```
#include<iostream>
using namespace std;
```

```
void prt( );
```

```
int i;
```

```
int main() {
    for(i = 0; i < 5; i++)
        prt( );
    return 0;
}
```

```
> D:\Edu-CPP\chap11\ex11-03.exe
```

```
*****
```

```
void prt( ) {
    for(i = 0; i < 5; i++)
        cout << '*';
    cout << '\n';
}
```



◆ (5) 命名空间作用域 (**全局变量**)

■ 命名空间的语法形式

■ 例如:

```
namespace SomeNs
{
    class SomeClass{ };
};
```

namespace 命名空间名

{

命名空间的各种声明 (函数声明、类声明、...)

};

C++标准程序库中所有标识符都定义于名为std的namespace

■ 使用SomeClass (以下定义类对象的方式是等价的)

① SomeNs::SomeClass obj1;

② using SomeNs::SomeClass; SomeClass obj2;

③ using namespace SomeNs; SomeClass obj3;

cin、cout、endl等在std命名空间中声明，使用的方式可以是：

① std::cin, std::cout, std::endl;

② using std:: cin;
 using std::cout;
 using std::endl;

③ #include<iostream>
 using namespace std;

示例：作用域/可见性



应尽量少使用全局变量，因为：

- ①全局变量在程序全部执行过程中占用存储单元；
- ②如果函数内使用全局变量，函数独立性变弱，降低了函数的通用性、可靠性，可移植性
- ③降低程序清晰性，增加关联性，容易出错

```
> D:\Edu-CPP\chap11\ex11-04.exe  
  
i = 7  
j = 6  
i = 5
```

```
#include <iostream>  
using namespace std;  
int i; 在全局命名空间中的全局变量  
namespace Ns { int j; } Ns命名空间中的全局变量  
int main() {  
    i = 5; 为全局变量i赋值  
    Ns::j = 6; 为全局变量j赋值  
    {  
        using namespace Ns; 在当前块中可以直接引用Ns命名空间的标识符  
        int i; 局部变量i  
        i = 7;  
        cout << "i = " << i << endl;  
        cout << "j = " << j << endl; Ns命名空间的全局变量j  
    }  
    cout << "i = " << i << endl; 全局变量i  
    return 0;  
}
```

本章主要内容



11.1 数据共享与保护：鱼和熊掌兼得

11.2 变量/对象的作用域与可见性（空间）

11.3 变量/对象的生存期（时间）



11.4 静态变量/类的静态成员

11.5 类的友元

11.6 共享数据的保护

11.7 多文件结构和编译预处理命令

11.8 作业与实践

变量的生存期



◆变量占用内存空间的时间段称为生存期，分为三类：静态生存期、动态生存期、自动生存期

- 全局变量具有静态生存期

- 局部变量和函数的参数**一般**具有自动生存期

- 用new运算符分配内存、用delete释放内存的动态变量具有动态生存期

静态变量声明形式：**static** 类型 变量名列表；
例如：**static** int a;
可以是全局变量，也可以是局部变量

◆定义局部变量时，可以加上**存储类型修饰符**：**auto**、**static**或**register**显式指出其生存周期

- 局部变量的**默认存储类型为auto**（常省略auto）：**int i;**实为**auto int i;**

- static**存储类型的局部变量具有静态生存期

- register**存储类型的局部变量也具有自动生存期，与**auto**存储类型的局部变量的区别在于**register**是**建议**编译器将相应的局部变量的空间分配在**CPU的存储器**中，目的是提高对局部变量的访问效率，当然，**register**类型的局部变量的存储空间也可以在**CPU的寄存器**中，或内存中

不同生存期变量的作用域/可见性



- ◆ 体会全局变量、局部变量、自动变量、静态变量的作用域与可见性
- ◆ 静态变量在程序开始时已分配存储单元，若没有初始化，默认初值0

```
int i = 1; 全局变量，静态生存期
```

```
void other()
```

```
{
```

```
    static int a = 2;
```

静态局部变量，只在第1次调用时初始化

```
    static int b;
```

```
    int c = 10;
```

局部变量，自动生存期

```
    a += 2; i += 32; c += 5;
```

```
    cout << "---other--";
```

```
    cout << "i:" << i << ", a:" << a
```

```
        << ", b:" << b << ", c:" << c << '\n';
```

```
    b = a;
```

```
}
```

```
int main() {
```

```
    static int a;
```

静态局部变量，静态生存期

```
    int b = -10, c = 0;
```

局部变量，自动生存期

```
    cout << "---main---";
```

```
    cout << "i:" << i << ", a:" << a
```

```
        << ", b:" << b << ", c:" << c << '\n';
```

```
    c += 8; other();
```

```
    cout << "---main---";
```

```
    cout << "i:" << i << ", a:" << a
```

```
        << ", b:" << b << ", c:" << c << '\n';
```

```
    i += 10; other();
```

```
    return 0;
```

```
}
```

D:\Edu-CPP\chap11\ex11-05.exe

```
---main---i:1, a:0, b:-10, c:0
---other--i:33, a:4, b:0, c:15
---main---i:33, a:0, b:-10, c:8
---other--i:75, a:6, b:4, c:15
```



变量的时间、空间特性

◆全局变量（静态）

- 时间和空间上，与程序同在

◆局部变量（静态）

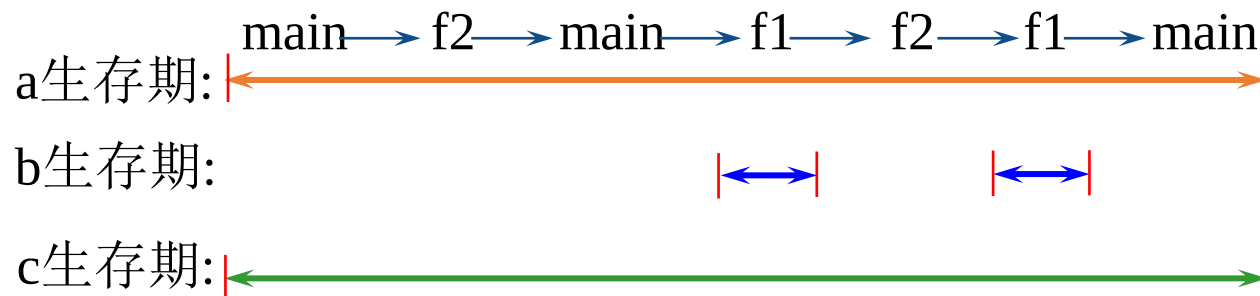
- 被封印在有限空间，时间不限，空间受限

◆局部变量（自动）

- 有限时间，有限空间

```
int a;  
int main() {  
    .....  
    f2;  
    .....  
    f1;  
    .....  
}  
f1() {  
    int b;  
    .....  
    f2;  
    .....  
}  
f2() {  
    static int c;  
    .....  
}
```

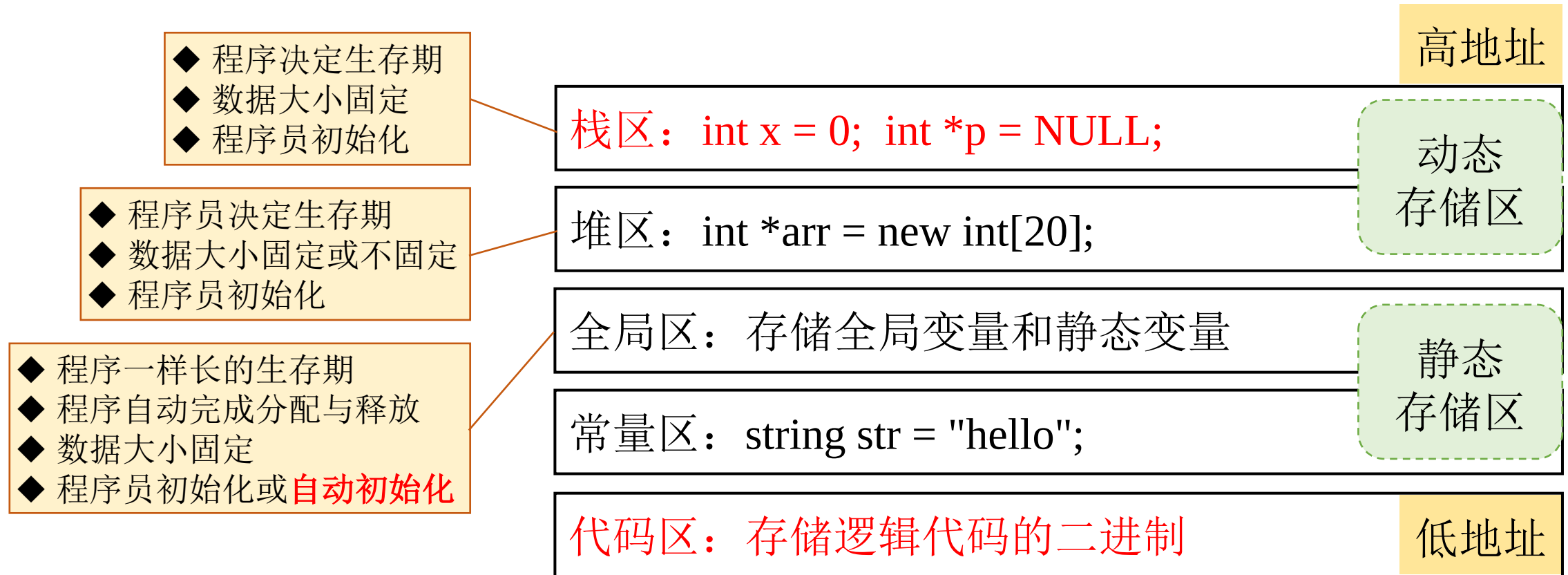
a作用域



b作用域

c作用域

程序的内存分区



本章主要内容



11.1 数据共享与保护：鱼和熊掌兼得

11.2 变量/对象的作用域与可见性（空间）

11.3 变量/对象的生存期（时间）

11.4 静态变量/类的静态成员



11.5 类的友元

11.6 共享数据的保护

11.7 多文件结构和编译预处理命令

11.8 作业与实践

类的静态成员



◆共享数据——全局变量

◆对于自定义的类，如何才能在类范围内共享数据呢？

- 利用类的静态成员

◆类的静态成员分为：静态数据成员和静态成员函数

- 作用：解决类不同对象间的**数据和函数共享的问题**，平衡共享和安全

◆静态数据成员

- 在类内用**static**关键字声明的数据成员，必须在类外定义和初始化，若不初始化，则值为0

 - 类外定义和初始化的形式为：**数据类型 类名::静态数据成员名 = 值;**

- 静态数据成员只有一份拷贝，被类的所有对象**共享**

- 静态数据成员具有静态生存期，编译时即确定内存

- 程序的其它部分对静态数据成员的访问，受它在类中声明的访问控制的限制

 - 类外不通过类的对象，public静态数据成员的一般用法是：**类名::标识符**

静态数据成员



```
class Demo
{
public:
    int x;
    static int y;
};
```

必须在类内声明，
类外定义和初始化

```
int Demo::y = 10;
```

Demo::y

10

静态变量

d1

x

d2

x

实例变量

```
int main()
```

```
{
```

```
    cout << "Demo::y = " << Demo::y << endl;
```

```
    Demo d1, d2;
```

```
    cout << "d1.y = " << d1.y << endl;
```

```
    cout << "d2.y = " << d2.y << endl;
```

```
    d1.y += 10;
```

```
    cout << "d1.y = " << d1.y << endl;
```

```
    cout << "d2.y = " << d2.y << endl;
```

```
    cout << "sizeof(d1)=" << sizeof(d1) << endl;
```

```
    return 0;
```

```
}
```

D:\Edu-CPP\chap11\ex11-06.exe

```
Demo::y = 10
```

```
d1.y = 10
```

```
d2.y = 10
```

```
d1.y = 20
```

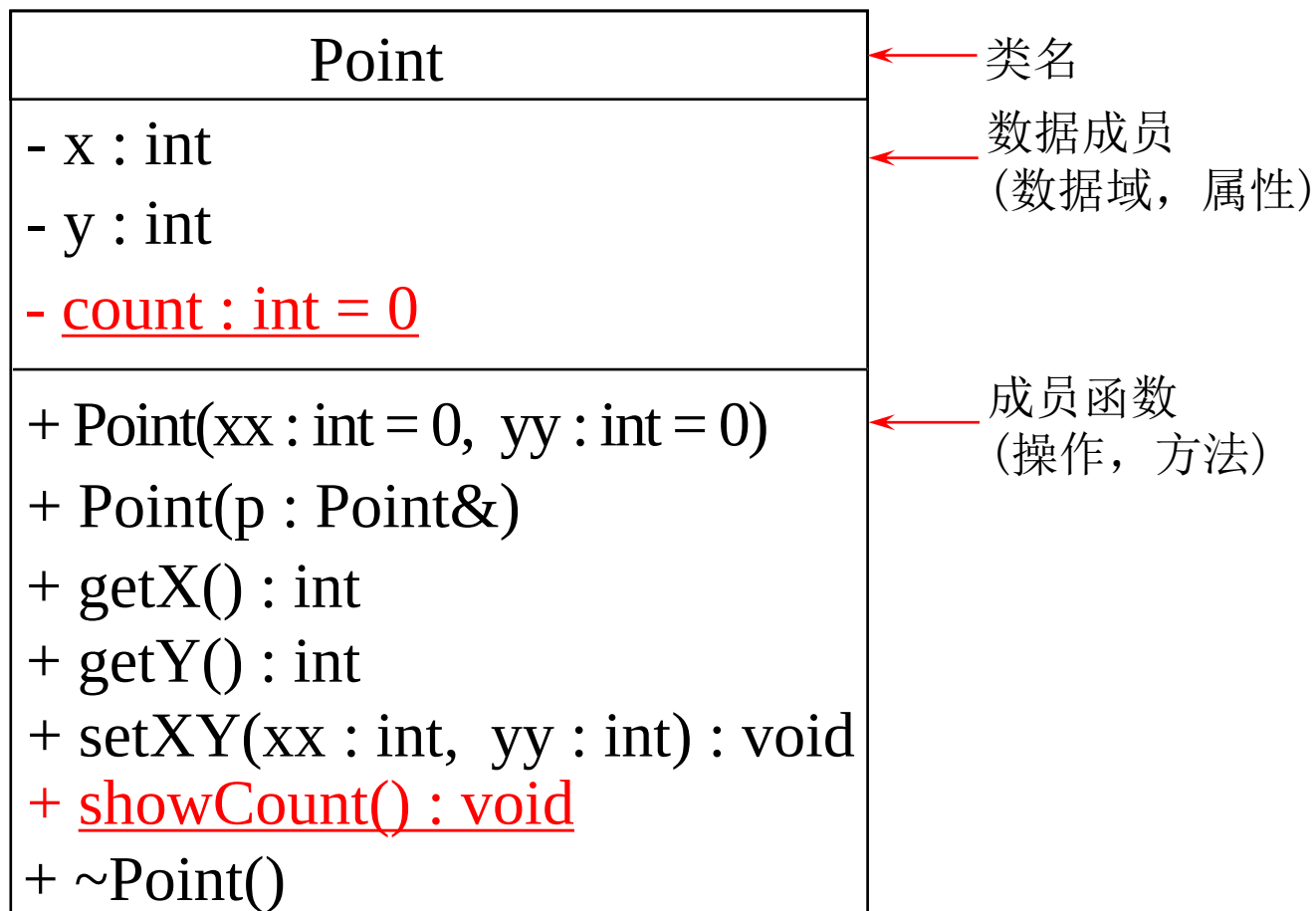
```
d2.y = 20
```

```
sizeof(d1)=4
```

示例：具有静态数据成员的Point类



- ◆ 设计一个简单Point类，功能包括：Point类对象初始化，显示点的x坐标和y坐标，记数Point类对象的个数



类的UML图示（UML类图）

UML（Unified Modeling Language, 统一建模语言）

UML规定属性的表示方式为：

可见性 名称 : 类型 [= 缺省值]

UML规定操作的表示方式为：

可见性 名称 (参数列表) [: 返回类型]

可见性（访问权限）标识符：

+表示public成员

-表示private成员

#表示protected成员

```
class Point {
```

```
public:
```

```
    Point(int x = 0, int y = 0) : x(x), y(y) {
```

```
        count++;
```

```
    }
```

```
    Point(Point &p) {
```

```
        x = p.x;  y = p.y;
```

```
        count++;
```

```
    }
```

```
    ~Point() { count--; }
```

```
    void setXY(int xx, int yy) {
```

```
        x = xx;  y = yy;
```

```
    }
```

```
    int getX() { return x; }
```

```
    int getY() { return y; }
```

```
    void showCount() {
```

```
        cout << "Object count = " << count << endl;
```

```
    }
```

```
private:
```

```
    int x, y;
```

```
    static int count;
```

```
};
```

```
int Point::count = 0;
```

构造函数中对count累加，
所有对象共同维护一个count

析构函数对count的维护

输出静态数据成员

静态数据成员声明，用于记录对象的个数

静态数据成员定义和初始化，使用类名限定

```

int main( )
{
    Point a(4, 5);
    cout << "Point A(" << a.getX() << ", " << a.getY() << "): ";
    a.showCount();
    Point b(a);
    cout << "Point B(" << b.getX() << ", " << b.getY() << "): ";
    b.showCount();
    return 0;
}

```

```

int main( )
{
    Point a(4, 5); 定义对象a, 构造函数使count增1
    a.showCount( );
    {
        Point p[5]; 定义对象数组, 构造函数使count增5
        a.showCount( );
    } 析构函数使count减5
    Point b(a); 定义对象b, 构造函数使count增1
    a.showCount( );
    return 0;
}

```

D:\Edu-CPP\chap11\ex11-07.exe

```

Point A(4, 5): Object count = 1
Point B(4, 5): Object count = 2

```

D:\Edu-CPP\chap11\ex11-07.exe

```

Object count = 1
Object count = 6
Object count = 2

```

静态成员函数

◆静态成员函数是使用static声明的成员函数，用于处理该类的静态数据

- 类外代码使用类名和作用域操作符调用静态成员函数
- 静态成员函数**没有this指针**，**只能**操作属于该类的静态的数据和函数成员
 - 静态成员函数不能使用非静态成员
 - 非静态成员函数可以使用静态成员

非静态成员函数的调用是通过对象调用的（知道哪个对象调用它；静态成员函数没有this指针，不知道哪个对象调用它）

```
> D:\Edu-CPP\chap11\ex11-07C.exe
```

```
Object count = 1
Object count = 2
```

```
int main( ) {
    Point a(4, 5);
    Point::showCount( );
    Point b(a);
    Point::showCount( );
    return 0;
}
```

```
class Point {
public:
    Point(int x = 0, int y = 0) : x(x), y(y) {
        count++;
    }
    Point(Point &p) {
        x = p.x; y = p.y;
        count++;
    }
    ~Point() { count--; }
    void setXY(int xx, int yy) {
        x = xx; y = yy;
    }
    int getX() { return x; }
    int getY() { return y; }
    static void showCount() { 静态成员函数
        cout << "Object count = " << count << endl;
    }
private:
    int x, y;
    static int count; 静态数据成员声明，
    用于记录对象的个数
};
int Point::count = 0;
```




```
class Test {
public:
    static void f(Test t);
    static void f2() {
        x++; // 错误，须通过对象名或引用
        y++; }
    void f3() { x--; y--; }
    void f4() { f2(); }
private:
    int x;
    static int y;
};

int Test::y = 0;

void Test::f(Test t) {
    cout << x; // 错误，歧义（静态成员函数没有this指针，不知是哪个对象的x）
    cout << t.x; // 正确
}
```

	静态成员函数 (调用：类外通过类名， 非静态成员函数内通过对象)	非静态成员函数 (调用：通过对象)
静态 数据成员	直接访问	直接访问
非静态 数据成员	通过对象名或引用 (也可以不设计为静态函数)	直接访问

```
int main()
{
    Test::f2();
    return 0;
}
```

本章主要内容



11.1 数据共享与保护：鱼和熊掌兼得

11.2 变量/对象的作用域与可见性（空间）

11.3 变量/对象的生存期（时间）

11.4 静态变量/类的静态成员

11.5 类的友元



11.6 共享数据的保护

11.7 多文件结构和编译预处理命令

11.8 作业与实践



类的友元概述

- ◆一般规则：只有类的成员函数才能访问类的私有成员
- ◆友元特权：让不是类的成员函数访问类的私有成员，被授予特别访问控制权
 - 规则不仅是用来遵守的，也是用来打破的
 - 友元避免把类成员全部设置成public，又让非类成员实现对类的访问，最大限度的保护数据成员的安全
 - 友元是C++提供的一种破坏封装和数据隐藏的机制，类外可以访问类的私有成员，**慎用**
 - 为了确保数据的完整性及数据封装与隐藏的原则，建议尽量不使用或少使用友元
- ◆通过声明友元，一个模块能够引用另一个模块中本被隐藏的信息
 - 友元函数
 - 友元类

友元函数



◆在类内声明一个普通函数时，前面加上**friend**修饰，那么该函数就成了该类的友元，可以通过对象名访问该类的任何成员

■友元函数有权访问该类所有成员，包括私有、保护和公有成员

```
#include<iostream>
#include<cmath>
using namespace std;

class Point
{
public:
    Point(int x = 0, int y = 0) : x(x), y(y) { }
    int getX() { return x; }
    int getY() { return y; }
    friend float dist(Point &a, Point &b);
private:
    int x, y;
};
```

```
float dist(Point& a, Point& b)
```

```
{
    double x = a.x - b.x;
    double y = a.y - b.y;
    return sqrt(x*x + y*y);
}
```

```
int main() {
    Point p1(1, 1), p2(4, 5);
    cout << "The distance is: ";
    cout << dist(p1, p2) << endl;
    return 0;
}
```

```
> D:\Edu-CPP\chap11\ex11-08.exe
```

```
The distance is: 5
```



◆ 友元函数不是类的成员函数，它只是被声明为类友元的普通函数

- 友元函数是在类声明中由关键字friend修饰，非成员函数，在友元函数的函数体中能够通过对象名访问private和protected成员
- 作用：增加灵活性，使程序员可以在信息封装和信息开放方面做合理安排

```
class Test
{
    int k;
    friend void func();
};

void func() { cout << 5; }

int main() {
    Test t;
    t.func();//对不对? ✗
    return 0;
}
```

```
int main()
{
    Test t;
    t.func();//对不对?
    return 0;
}
```

Compile Log ✓ Debug 🔍 Find Results ✗

Message

In function 'int main()':

[Error] 'class Test' has no member named 'func'



友元类

◆ 友元类是指将一个类声明为另一个类的友元

- 在实现类之间数据共享时，减少系统开销，提高效率

◆ 友元关系是单向的

- 如果声明类B是类A的友元，**类B的成员函数可以访问类A的私有和保护数据**，反之不行

➢ 类B的成员函数都是A类的友元函数

◆ 友元关系不具有继承性和传递性

◆ 友元不是当前类的成员，声明可以放在公有部分或私有部分

```
class B; // 前向声明
```

```
class A {  
    friend class B;  
public:  
    void display() {  
        cout << x << endl;  
    }  
private:  
    int x;  
};
```

```
class B {  
public:  
    void set(int i);  
    void display();  
private:  
    A a;  
};  
  
void B::set(int i) {  
    a.x = i;  
}  
  
void B::display() {  
    a.display();  
}
```

本章主要内容



11.1 数据共享与保护：鱼和熊掌兼得

11.2 变量/对象的作用域与可见性（空间）

11.3 变量/对象的生存期（时间）

11.4 静态变量/类的静态成员

11.5 类的友元

11.6 共享数据的保护



11.7 多文件结构和编译预处理命令

11.8 作业与实践

常对象



- ◆数据隐藏保护数据安全性
 - ◆数据共享破坏数据安全性
- ✓既共享又防止改变——常量数据(const)
✓常对象、常成员和常引用

◆const对象

- 定义形式：**const 类名 对象名**
- 必须进行初始化，不能被更新

```
class A
{
public:
    A(int i, int j) {x = i; y = j;}
    ...
private:
    int x, y;
};
const A a(3, 4);
```




常成员

◆ const类成员：用const修饰的类成员

- const成员函数

- const数据成员

◆ Const成员函数（只读成员函数）

- 只读函数不能修改类的数据成员

- 只能调用const成员函数，不能调用非const成员函数

```
class Screen{  
    int i;  
public:  
    void set(int val){ i = val; }  
    int get() const; // int get() const { return i; }  
};  
  
int Screen::get() const { return i; }
```

在类外定义时，必须加上const关键字

```
int Screen::get( ) const{ return ++i; } ❌
```

```
int Screen::get( ) const {  
    set(0); ❌  
    return i;  
}
```

const成员函数只能读取数据成员或调用其他const成员函数；不能更改数据成员或调用非const成员函数



◆ const对象与const成员函数示例

```
class Demo {  
public:  
    void f() {  
        cout << "non const" << endl;  
    }  
    void f() const {  
        cout << "const" << endl;  
    }  
};  
int main() {  
    Demo a;  
    a.f();  
    const Demo b;  
    b.f();  
    return 0;  
}
```

const可以重载函数

类对象的常量性决定调用哪个版本的成员函数

const对象只能调用const成员函数，必须提供一个const版本的成员函数

D:\Edu-CPP\chap11\ex11-10.exe

non const
const



◆ const数据成员

■ 定义及初始化示例:

```
class A {  
public:  
    A(int size) : SIZE(size) { };  
private:  
    const int SIZE;  
};
```

◆ const数据成员初始化

■ 必须且只能在构造函数**初始化列表**中初始

■ 不能在构造函数函数体内初始化，不能在定义处初始化

任何函数都不能对常数据成员赋值

```
class A {  
public:  
    A(int i) : a(i) { };  
    void print( ) const{ cout << a << ":" << b << endl; }  
private:  
    const int a;  
    static const int b; // 存储类型优先  
};  
  
const int A::b = 10;  
  
int main() {  
    A a1(100), a2(0);  
    a1.print();  
    a2.print();  
    return 0;  
}
```



D:\Edu-CPP\chap11\ex11-11.exe

100:10
0:10



常引用

- ◆ **const** 引用语法格式: **const** 类型说明符 & 引用名
- ◆ 常引用所引用的对象不能被更新
- ◆ 如果用常引用做形参, 便不会意外发生对实参的更改

```
class Point
{
public:
    Point(int x = 0, int y = 0) : x(x), y(y) { }
    int getX() { return x; }
    int getY() { return y; }
    friend float dist(const Point &a, const Point &b);
private:
    int x, y;
};
```

```
float dist(const Point& a, const Point& b)
{
    double x = a.x - b.x;
    double y = a.y - b.y;
    return sqrt(x*x + y*y);
}

int main() {
    Point p1(1, 1), p2(4, 5); // const Point p1(1, 1), p2(4, 5);
    cout << "The distance is: ";
    cout << dist(p1, p2) << endl;
    return 0;
}
```

```
> D:\Edu-CPP\chap11\ex11-12.exe
```

```
The distance is: 5
```

本章主要内容



11.1 数据共享与保护：鱼和熊掌兼得

11.2 变量/对象的作用域与可见性（空间）

11.3 变量/对象的生存期（时间）

11.4 静态变量/类的静态成员

11.5 类的友元

11.6 共享数据的保护

11.7 多文件结构和编译预处理命令

11.8 作业与实践



C++程序的一般组织结构

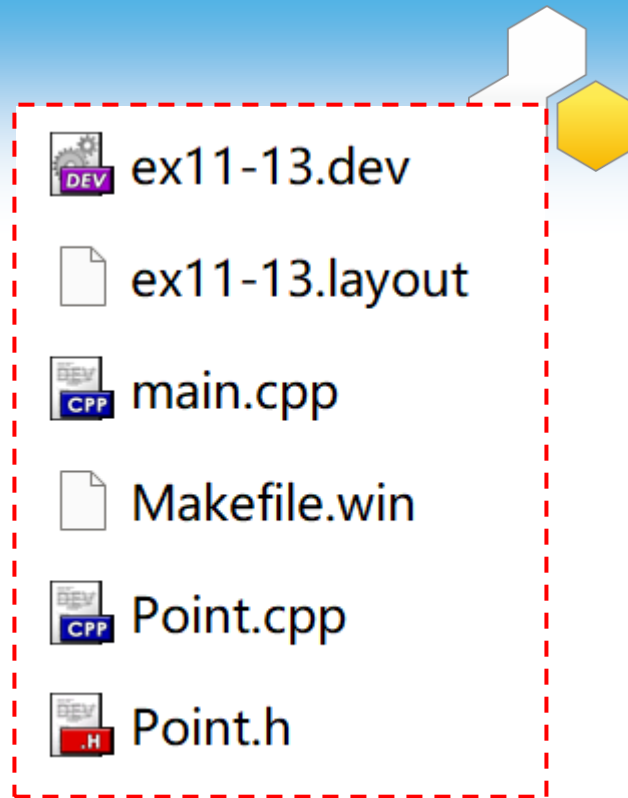
◆ 一个源程序可以划分为多个源文件

- 类声明文件（.h文件）
- 类实现文件（.cpp文件）
- 类的使用文件（main()所在的.cpp文件）

◆ 利用工程来组合各个文件

◆ 优点

- 把类的定义与实现分离开来，便于文档管理、维护
- 可将类的实现隐藏起来，使软件开发商能独立开发软件（发行.h和二进制文件）
- 便于团队式的大型软件开发



```
#ifndef POINT_H
#define POINT_H
```

文件Point.h, 类的定义

```
class Point {
public:
    Point(int x=0, int y=0) : x(x), y(y) {
        count++;
    }
    Point(const Point &p) : x(p.x), y(p.y) {
        count++;
    }
    ~Point() { count--; }
```

```
int getX() const { return x; }
int getY() const { return y; }
static void showCount();
```

```
private:
    int x, y;
    static int count;
};
#endif
```

D:\Edu-CPP\chap11\ex11-13\ex11-13.exe

```
Point A:4,5
Object count = 1
Point B:4,5
Object count = 2
```

```
#include "Point.h"
```

文件Point.cpp, 类的实现

```
#include<iostream>
using namespace std;
```

```
int Point::count = 0;
```

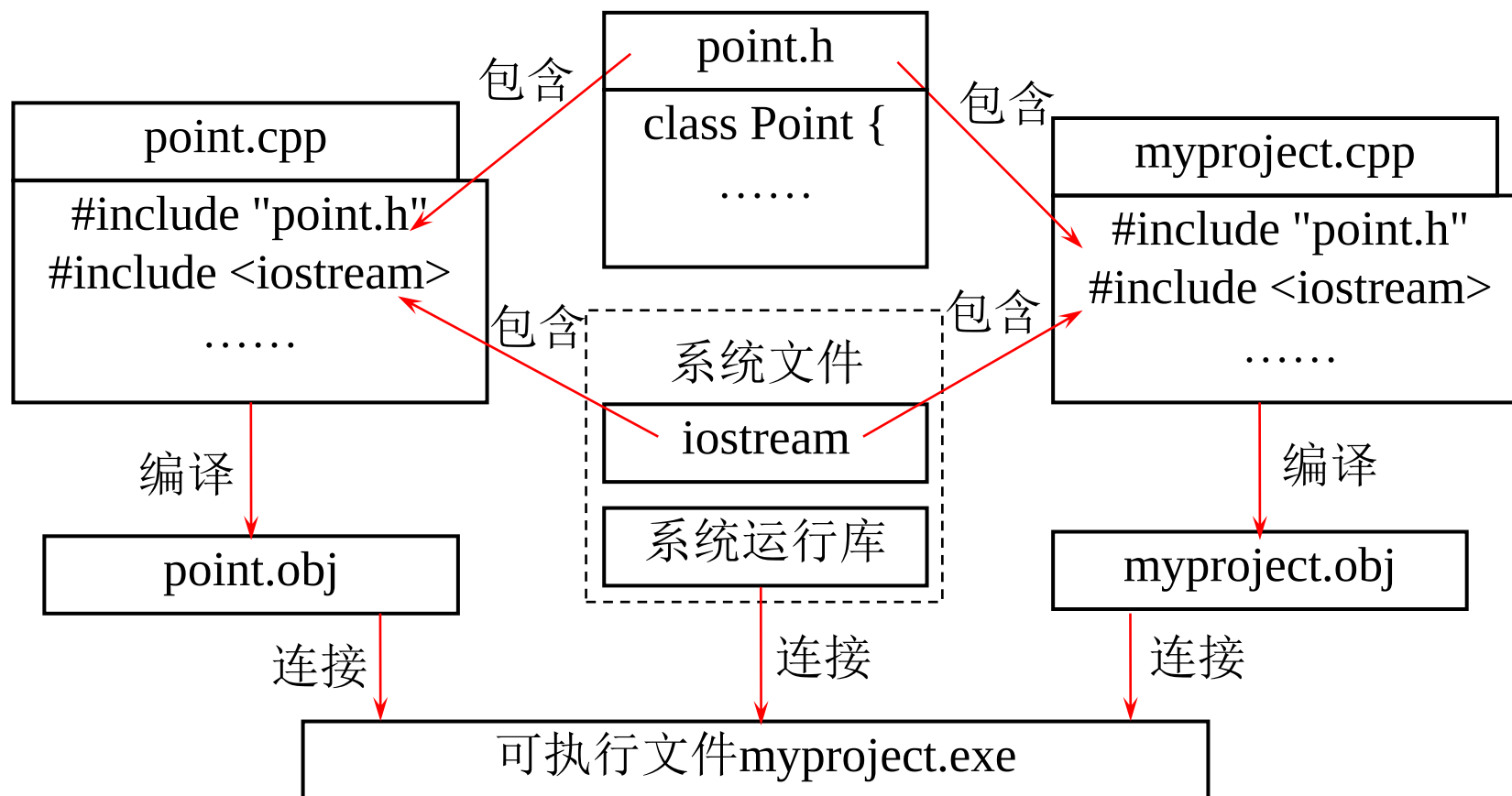
```
void Point::showCount() {
    cout << "\rObject count = " << count << endl;
}
```

```
#include "Point.h"
```

文件main.cpp, 实现主函数

```
#include<iostream>
using namespace std;
```

```
int main() {
    Point a(4, 5);
    cout << "Point A:" << a.getX() << "," << a.getY();
    Point::showCount();
    Point b(a);
    cout << "Point B:" << b.getX() << "," << b.getY();
    Point::showCount();
    return 0;
}
```



C++程序设计的一般步骤是：编辑（编写/修改代码）→编译→连接→运行

外部变量与外部函数



◆ 外部变量

- C++程序是多文件的组织结构，变量在一个文件中定义，其它文件中使用
- 文件作用域中定义的变量是外部变量，在其它文件中用**extern**关键字声明

◆ 外部函数

- 类外声明的函数（非成员函数），具有文件作用域
- 可在不同的编译单元中被调用，只要在调用之前进行引用性声明

```
Project  Classes  Debug  MyProject.cpp  MyProject.cpp
1  #include <iostream>
2  using namespace std;
3
4  int i = 3;
5  extern void other( );
6  void next( );
7
8  int main( )
9  {
10     i++;
11     next( );
12     return 0;
13 }
14
15 void next( )
16 {
17     cout << "in next( ): i = " << i << endl;
18     i++;
19     other( );
20 }
```

```
other.cpp  other.cpp
1  #include <iostream>
2  using namespace std;
3
4  extern int i;
5
6  void other( )
7  {
8     cout << "in other( ): i = " << i << endl;
9     i++;
10 }
```

D:\Edu-CPP\chap11\ex11-14\ex11-14.exe

```
in next( ): i = 4
in other( ): i = 5
```



编译预处理

◆ #include 包含指令

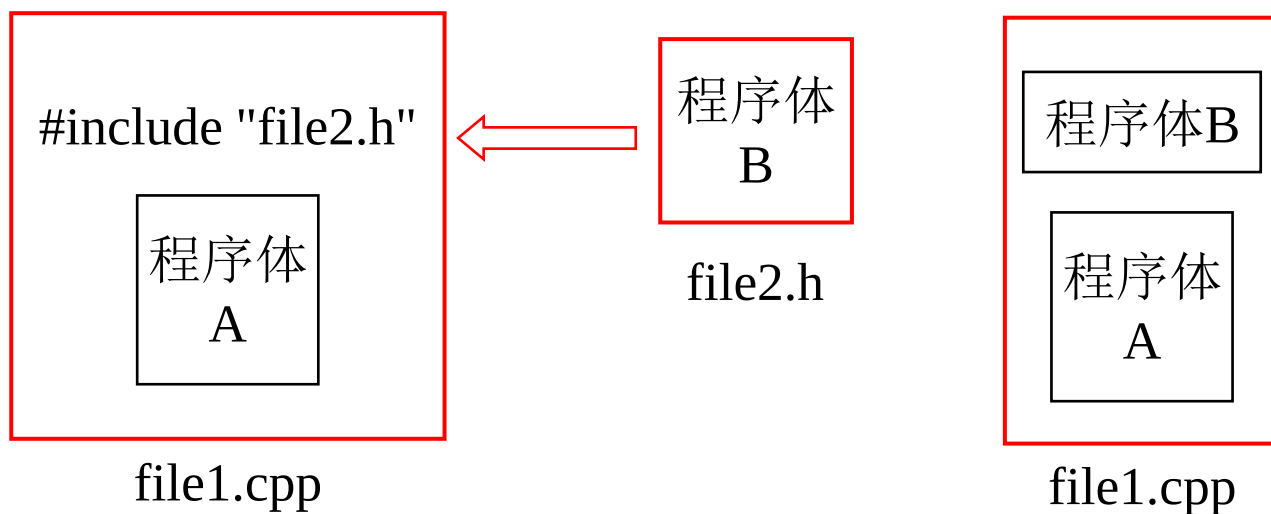
■ 将一个源文件嵌入到当前源文件中该点处

■ #include <文件名>

➢ 按标准方式搜索，文件位于C++系统目录的include子目录下

■ #include "文件名"

➢ 首先在当前目录中搜索，若没有，再按标准方式搜索



预编译时，用被包含文件的内容取代该预处理命令，再对“包含”后的文件作一个源文件编译



条件编译

◆通过条件控制编译器编译不同程序段，提高程序的移植性并方便调试

```
#if 常量表达式
    程序正文
#endif
.....
```

```
#if 常量表达式
    程序正文1
#else
    程序正文2
#endif
```

```
#if 常量表达式1
    程序正文1
#elif 常量表达式2
    程序正文2
#else
    程序正文3
#endif
```

```
#ifdef 标识符
    程序段1
#else
    程序段2
#endif
```

```
#ifndef 标识符
    程序段1
#else
    程序段2
#endif
```

通过条件编译选择求最大值或最小值

```
#define MAX

int main(void){
    int a, b;
    cout << "输入两整数:" ;
    cin >> a >> b;
#ifdef MAX
    cout << (a >= b ? a : b);
#else
    cout << (a <= b ? a : b);
#endif
    return 0;
}
```



```
int main(void){
    int a, b;
    cout << "输入两整数:" ;
    cin >> a >> b;
    cout << (a >= b ? a : b);
}
```

D:\Edu-CPP\chap11\ex11-15.exe

输入两整数:20 30
30



◆防止变量的重复定义

```
#ifndef PI
#define DOUBLEPI 3.14*2
#else
#define DOUBLEPI PI*2
#endif
```

◆预处理最主要目的是防止头文件的重复包含和编译

```
#ifndef _CLOCK_H
#define _CLOCK_H

class Clock {
public:
    void setTime(int h=0, int m=0, int s=0);
    void showTime();
private:
    int hour, minute, second;
};
#endif
```

本章主要内容



11.1 数据共享与保护：鱼和熊掌兼得

11.2 变量/对象的作用域与可见性（空间）

11.3 变量/对象的生存期（时间）

11.4 静态变量/类的静态成员

11.5 类的友元

11.6 共享数据的保护

11.7 多文件结构和编译预处理命令

11.8 作业与实践





◆（已归并到第10章作业中）